Sentiment Analysis with Text Mining with R

AUTHOR
Pascal Hermann Kouogang Tafo

INTRODUCTION

Human often use their understanding of the emotional intent of words to infer whether a section of text is positive or negative. In Chapter 2 of Text Mining with R,authors introduce sentiment analysis and our assignment consists to reproduce and extend the primary example.

PLANNED APPROACH

To tackle this task, we will go work as followed:

- Load the **tidyverse**, **tidytext**, and **janeaustenr** libraries to mirror the original environment.
- Process the text of *Emma* and *Pride and Prejudice* using the **bing** and **nrc** lexicons to recreate the net sentiment trajectories.
- Import a new dataset to test the flexibility of the tidy format.
- Incorporate a third lexicon to observe how domain-specific emotional tagging differs from general-purpose dictionaries.
- Visualize the results of all three lexicons against the new corpus to identify shifts in absolute vs. relative sentiment.

Step 1: Reproducing the base example Analysis

In this section, we will focus on recreating the original sentiment analysis of Jane Austen's novels using the **bing** lexicon as described by Silge and Robinson

Let's Install and upload the necessary libraries

```
library(tidyverse)
```

```
Warning: package 'tidyverse' was built under R version 4.5.2

Warning: package 'tibble' was built under R version 4.5.2

Warning: package 'tidyr' was built under R version 4.5.2

Warning: package 'readr' was built under R version 4.5.2
```

```
Warning: package 'purrr' was built under R version 4.5.2

Warning: package 'dplyr' was built under R version 4.5.2

Warning: package 'stringr' was built under R version 4.5.2

Warning: package 'forcats' was built under R version 4.5.2

Warning: package 'lubridate' was built under R version 4.5.2
```

```r
library(tidytext)
```

```
Warning: package 'tidytext' was built under R version 4.5.3
```

```r
library(janeaustenr)
```

```
Warning: package 'janeaustenr' was built under R version 4.5.3
```

```r
library(stringr)
```

Let's Recreate the net sentiment trajectory for Emma and Pride & Prejudice

In order to reproduce the primary example code from chapter 2 which in our case is the net sentiment trajectory for Emma and Pride & Prejudice, we will use the foundational "tidying" steps required to transform raw text into a format suitable for sentiment analysis using tidy data principles. I use Gemini to help create a Qmd file syntax that runs primary example code from chapter 2.

```r
# Tidy the Jane Austen books

tidy_books <- austen_books() %>%
  group_by(book) %>%
  mutate(
    linenumber = row_number(),
    chapter = cumsum(str_detect(text, regex("^chapter [\\divxlc]", ignore_case = TRUE)))
  ) %>%
  ungroup() %>%
  unnest_tokens(word, text)

head(tidy_books,10)
```

```
# A tibble: 10 × 4
   book                linenumber chapter word
   <fct>                    <int>   <int> <chr>
 1 Sense & Sensibility          1       0 sense
 2 Sense & Sensibility          1       0 and
 3 Sense & Sensibility          1       0 sensibility
```

```
 4 Sense & Sensibility          3       0 by
 5 Sense & Sensibility          3       0 jane
 6 Sense & Sensibility          3       0 austen
 7 Sense & Sensibility          5       0 1811
 8 Sense & Sensibility         10       1 chapter
 9 Sense & Sensibility         10       1 1
10 Sense & Sensibility         13       1 the
```

```
# Recreate the net sentiment trajectory for Emma and Pride & Prejudice

austen_sentiment <- tidy_books %>%
  filter(book %in% c("Emma", "Pride & Prejudice")) %>%
  inner_join(get_sentiments("bing")) %>%
  count(book, index = linenumber %/% 80, sentiment) %>%
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>%
  mutate(sentiment = positive - negative)
```

```
Joining with `by = join_by(word)`

Warning in inner_join(., get_sentiments("bing")): Detected an unexpected many-to-many
relationship between `x` and `y`.
ℹ Row 155017 of `x` matches multiple rows in `y`.
ℹ Row 2497 of `y` matches multiple rows in `x`.
ℹ If a many-to-many relationship is expected, set `relationship =
  "many-to-many"` to silence this warning.
```

```
head(austen_sentiment,10)
```

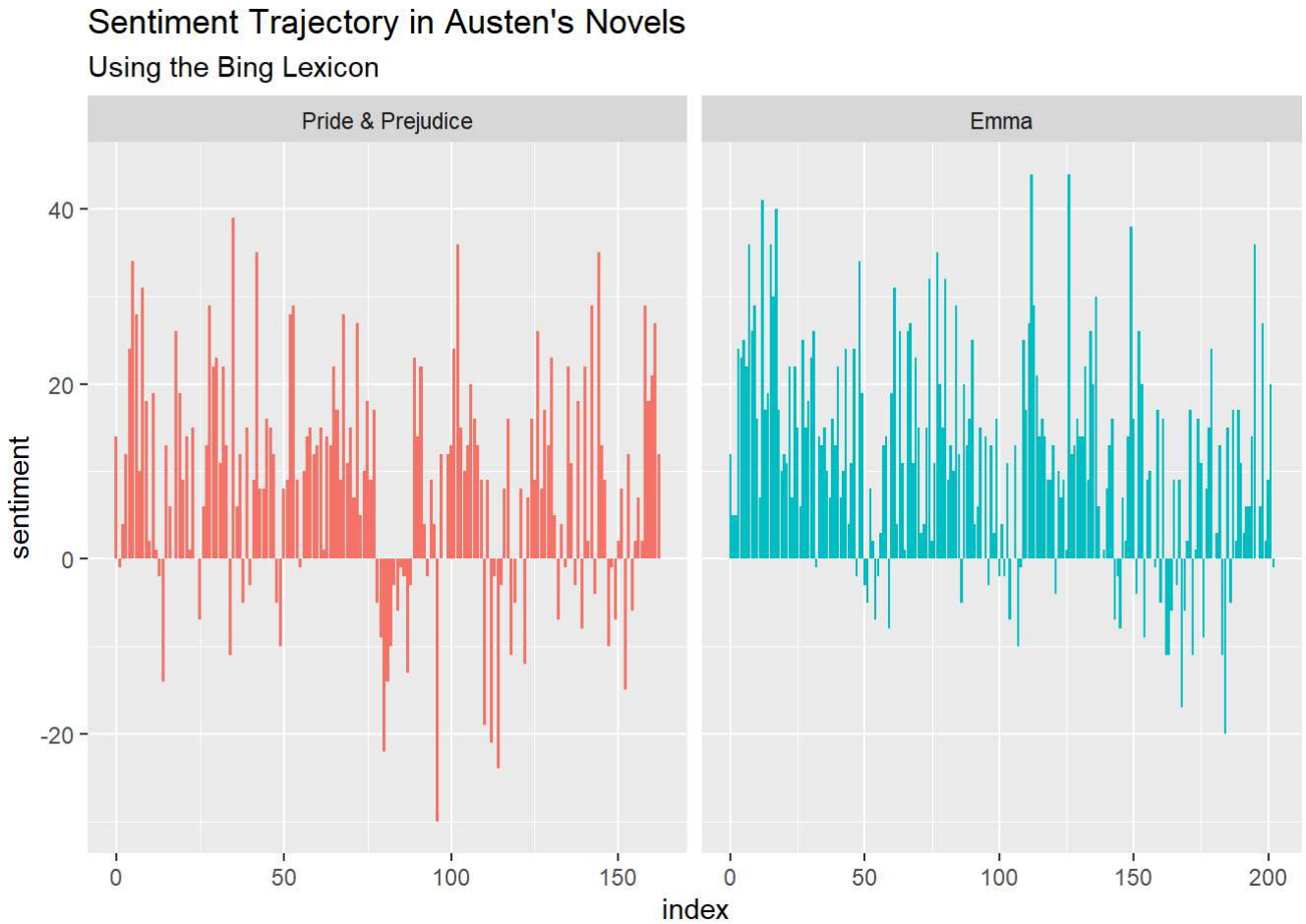
```
# A tibble: 10 × 5
   book              index negative positive sentiment
   <fct>             <dbl>    <int>    <int>     <int>
 1 Pride & Prejudice     0        7       21        14
 2 Pride & Prejudice     1       20       19        -1
 3 Pride & Prejudice     2       16       20         4
 4 Pride & Prejudice     3       19       31        12
 5 Pride & Prejudice     4       23       47        24
 6 Pride & Prejudice     5       15       49        34
 7 Pride & Prejudice     6       18       46        28
 8 Pride & Prejudice     7       23       33        10
 9 Pride & Prejudice     8       17       48        31
10 Pride & Prejudice     9       22       40        18
```

Let’s Visualize the original trajectory

We will visualize the original trajectory using **the Bing Lexicon**.

```
# Visualize the original trajectory
```

```r
ggplot(austen_sentiment, aes(index, sentiment, fill = book)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~book, ncol = 2, scales = "free_x") +
  labs(title = "Sentiment Trajectory in Austen's Novels",
       subtitle = "Using the Bing Lexicon")
```



Step 2: Extend Analysis

This second will consist at extending the analysis of the original example conducted above following two specifics ways.

1) Let's extend the analysis using a

To successfully reach our goal and text the flexibility of our analysis, we will analyze a collection of NYT news articles by creating a simulation of a small news corpus regarding soccer analytics.

```r
# Simulating a small news corpus regarding soccer analytics

NYT_news_data <- tibble(
  article = c(rep("Financial Report", 3), rep("Match Review", 3)),
  text = c(
```

```
    "The club reported a disastrous financial quarter with massive losses.",
    "Investors are worried about the debt and the failing market strategy.",
    "The board expressed deep regret over the poor fiscal performance.",
    "The young striker scored a brilliant goal in a spectacular victory.",
    "Fans are delighted with the team's creative and dominant playstyle.",
    "It was an amazing, glorious afternoon for the championship leaders."
  )
)

NYT_news_data
```

```
# A tibble: 6 × 2
  article          text
  <chr>            <chr>
1 Financial Report The club reported a disastrous financial quarter with massiv…
2 Financial Report Investors are worried about the debt and the failing market …
3 Financial Report The board expressed deep regret over the poor fiscal perform…
4 Match Review     The young striker scored a brilliant goal in a spectacular v…
5 Match Review     Fans are delighted with the team's creative and dominant pla…
6 Match Review     It was an amazing, glorious afternoon for the championship l…
```

```
Tidy_news <- NYT_news_data %>%
  group_by(article) %>%
  mutate(line = row_number()) %>%
  ungroup() %>%
  unnest_tokens(word, text)

head(Tidy_news,20)
```

```
# A tibble: 20 × 3
   article           line word
   <chr>            <int> <chr>
 1 Financial Report     1 the
 2 Financial Report     1 club
 3 Financial Report     1 reported
 4 Financial Report     1 a
 5 Financial Report     1 disastrous
 6 Financial Report     1 financial
 7 Financial Report     1 quarter
 8 Financial Report     1 with
 9 Financial Report     1 massive
10 Financial Report     1 losses
11 Financial Report     2 investors
12 Financial Report     2 are
13 Financial Report     2 worried
14 Financial Report     2 about
15 Financial Report     2 the
16 Financial Report     2 debt
17 Financial Report     2 and
18 Financial Report     2 the
```

```
19 Financial Report     2 failing
20 Financial Report     2 market
```

Interpretation

The resulting **Tidy_news** object is a long-form data frame that breaks down the unstructured news text into individual, analyzable units while maintaining the metadata of which article and line each word came from. This structure allows us to programmatically compare the highly negative vocabulary of the financial reports such as **disastrous** or **losses** against the positive language of the match reviews including **brilliant** or **victory** using **sentiment joins**. Essentially, it prepares the data so that the emotional **sum** of each article type can be calculated and visualized.

---

2) Let's add the AFINN Lexicon to extend the Analysis.

Here, we will incorporate the AFINN sentiment Lexicon, which provides a numeric ratings between -5 and 5 for each word, allowing a more detailed emotional assessment than the simple "yes/no" or positive/negative categorization found in binary lexicons such as Bing and NRC. Our analysis will then consists of comparing the results on the NYT news articles corpus from the original base example.

a) Comparison of Bing, NRC, and AFINN on the new corpus

```
# Let's install  and load the textdata package to download the AFINN lexicon and access this datas

#We will install the "textdata" package in our environment to easily render the file.

library(textdata)
```

```
Warning: package 'textdata' was built under R version 4.5.3
```

```
# AFINN: Numeric lexicon

news_afinn <- Tidy_news %>%
  inner_join(get_sentiments("afinn")) %>%
  group_by(article) %>%
  summarise(sentiment = sum(value)) %>%
  mutate(method = "AFINN")
```

```
Joining with `by = join_by(word)`
```

```
# Bing: Binary lexicon

news_bing <- Tidy_news %>%
  inner_join(get_sentiments("bing")) %>%
  count(article, sentiment) %>%
```

```
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>%
  mutate(sentiment = positive - negative, method = "Bing")
```

```
Joining with `by = join_by(word)`
```

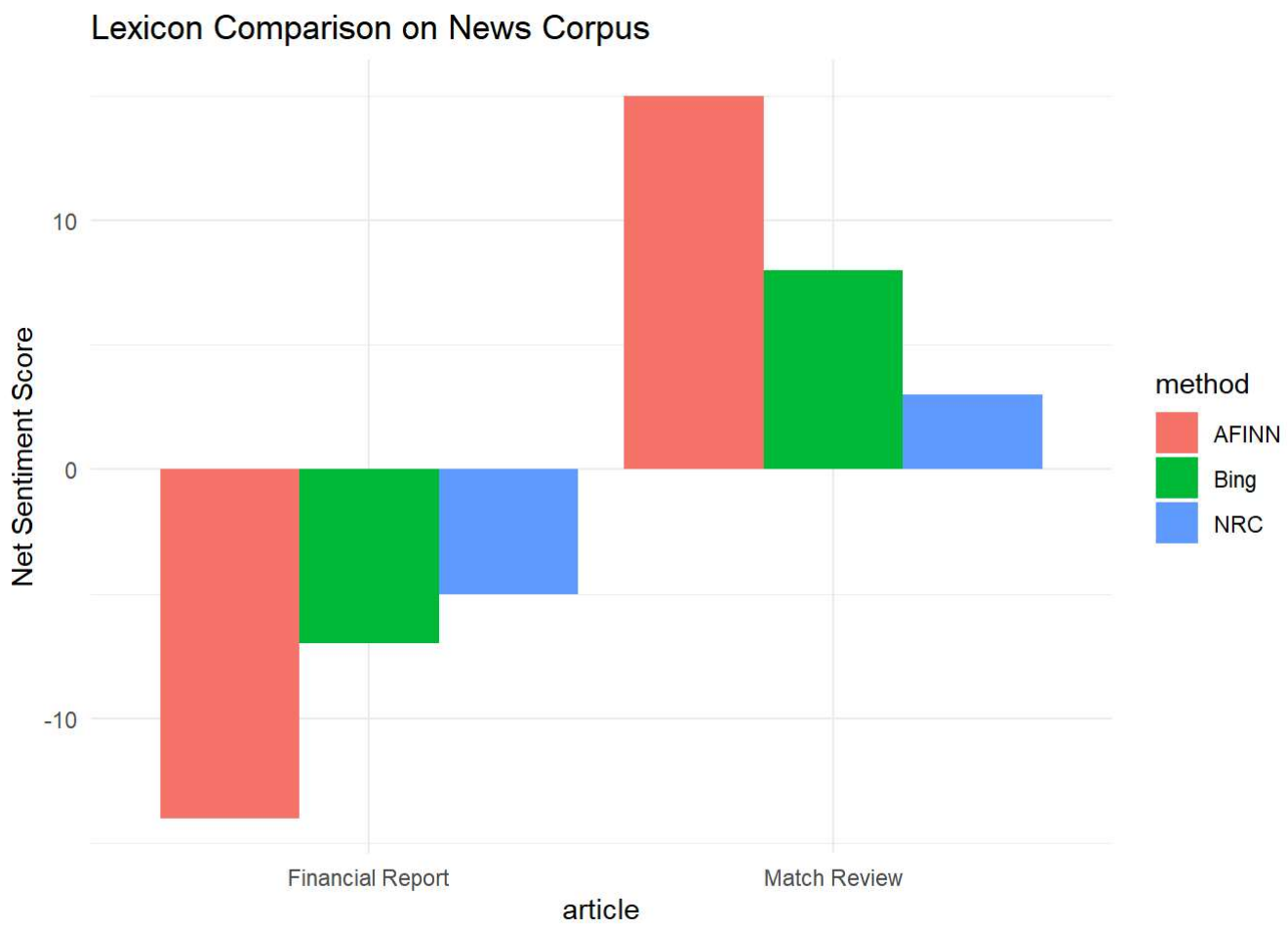
```
# NRC: Binary/Emotion-based lexicon

news_nrc <- Tidy_news %>%
  inner_join(get_sentiments("nrc") %>%
              filter(sentiment %in% c("positive", "negative"))) %>%
  count(article, sentiment) %>%
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>%
  mutate(sentiment = positive - negative, method = "NRC")
```

```
Joining with `by = join_by(word)`
```

b) Let's bind and Visualize

We will visualize a lexicon comparison

```
bind_rows(news_afinn, news_bing, news_nrc) %>%
  ggplot(aes(article, sentiment, fill = method)) +
  geom_col(position = "dodge") +
  theme_minimal() +
  labs(title = "Lexicon Comparison on News Corpus",
       y = "Net Sentiment Score")
```



Interpretation

As observed in the original text, when comparing these tools in text, we see that while they usually agree on whether a story is getting **happier or sadder**, they **measure** that feeling differently. In fact:

- **AFINN** Shows the highest variance by the scoring method because it gives extra weight to very strong words like **disastrous** or **amazing** and its results show much bigger jumps and drops than the others.
- **Bing** Often results in lower absolute scores because it is a straightforward **"yes/no"** system that counts every positive word as +1 and every negative word as -1 regardless of the **strength of the word**.
- **NRC** tends to be biased higher relative to the other two because its dictionary contains a higher percentage of positive words compared to negative ones when compared to the Bing list which is the other binary lexicon.

CONCLUSION

By completing this assignment, we can comfortably say that if we were tracking a movie's plot, all three lexicons (**AFINN, Bing, NRC**) would show us the same ups and downs. However, the **AFINN** lexicon would make the peaks look like mountains, while **Bing** would make them look like small hills.